



PRÉ-RENTRÉE 2026-2027

Dossier individuel

Ahmad BELDI - Première NSI

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

CONFIDENTIEL

1. Synthèse du diagnostic

- 13 questions traitées sur 18 ;
- aucun domaine encore totalement stabilisé ;
- Web : meilleur point d'appui ;
- représentation binaire et données en tables : réussites hésitantes ;
- réseaux : niveau à situer ;
- programmation : priorité principale, réussite 22,2 %.

Conceptions à reconstruire

- `b = a` copie la valeur, sans lien dynamique ultérieur ;
- `x = x + 1` est une affectation ;
- négation de `x > 5` : `x <= 5` ;
- `range(3)` produit trois valeurs ;
- l'indexation commence à 0 ;
- `len(L)` est un nombre d'éléments ;
- un paramètre n'est pas la valeur renvoyée ;
- sans `return`, une fonction renvoie `None`.

2. Plan d'action

Séance	Objectif personnel	Aide prévue	Indicateur
1	construire le modèle d'affectation	table de trace	4 traces exactes sur 5
2	maîtriser <code>range</code> , compteur et somme	valeurs de <code>range</code> écrites	boucle sans erreur de borne
3	distinguer paramètre, retour et <code>None</code>	gabarit de fonction	3 fonctions autonomes
4	stabiliser indexation et <code>len</code>	schéma indices/valeurs	5 accès corrects
5	assembler un projet	squelette détaillé	projet exécuté et expliqué

3. Suivi

Compétence	Initial	S1	S2	S3	S4	S5	Final
Affectation	1						
Conditions	2						
Boucles	1						
Fonctions	1						
Listes	1						
Tables CSV	2						
Autonomie	1						

4. Conseils pour l'année

- tracer à la main avant d'exécuter ;
- conserver un carnet d'erreurs ;
- programmer trois fois par semaine, même quinze minutes ;
- écrire deux tests par fonction ;
- ne pas copier un code sans pouvoir expliquer chaque variable ;
- revoir le mémento après chaque chapitre.

5. Modèle de bilan final

Recommandations de septembre



PRÉ-RENTRÉE 2026-2027

Remédiation ciblée

Ahmad BELDI - Python

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document élève

Affectations

1. Tracer `a=4; b=a; a=a+2; c=a+b`.
2. Expliquer pourquoi `b` ne change pas à la troisième ligne.
3. Corriger `age="16"; age=age+1`.

Conditions

1. Écrire la négation de `x < 4`.
2. Écrire la négation de `(x >= 0) and (x <= 10)`.
3. Tester les bornes d'un programme qui accepte les âges de 12 à 17 inclus.

Boucles

1. Écrire les valeurs produites par `range(4)`.
2. Écrire les valeurs produites par `range(1, 8, 2)`.
3. Compléter une somme de 1 à 5.
4. Corriger un compteur réinitialisé dans la boucle.

Listes

1. Pour `L=[3, 7, 9]`, donner `L[0]`, `L[1]`, `len(L)`, dernier indice.
2. Ajouter 12 à la liste.
3. Écrire une boucle affichant chaque valeur.

Fonctions

1. Identifier paramètre, argument et valeur renvoyée dans `carre(5)`.
2. Modifier une fonction qui affiche une moyenne pour qu'elle la renvoie.
3. Expliquer le résultat d'une fonction sans `return`.
4. Écrire trois assertions pour `est_positif`.

Bilan
